

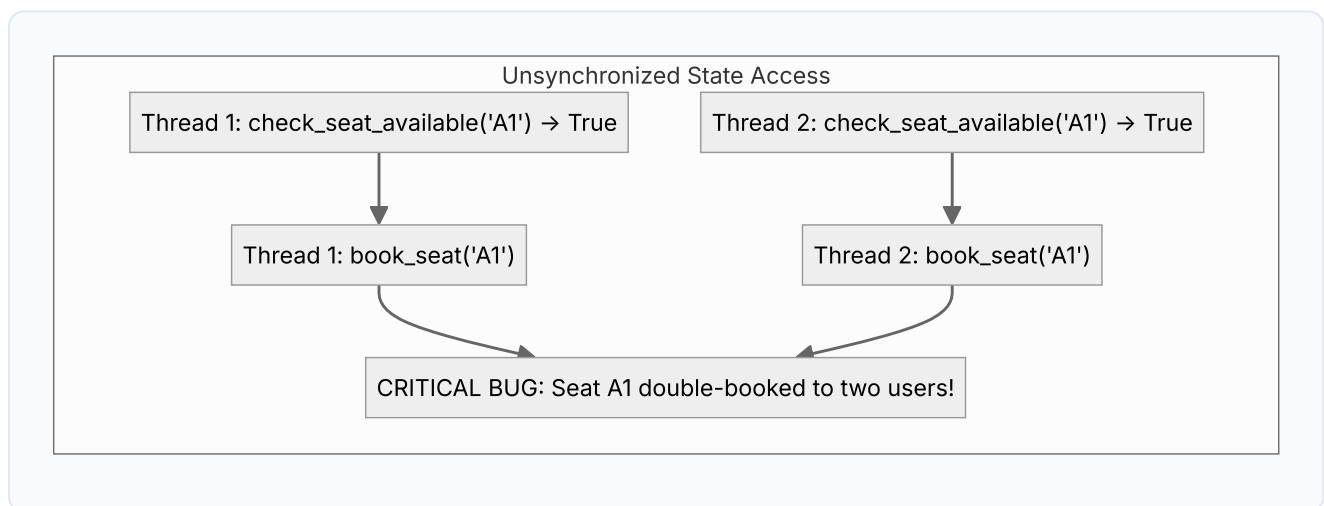
LLD Chapter 4: Concurrency Patterns, Thread-Safety & Locking in System Design

Core Learning Objective: Master concurrency management in Low-Level Design interviews. Learn how to write thread-safe systems, implement a Readers-Writer lock from scratch, prevent deadlocks, and use double-checked locking in Python.

1. Why Concurrency Matters in LLD Interviews

In real-world Product-Based Company interviews, **anyone can write an object-oriented class diagram**. The true differentiator between a junior engineer and a Senior/Staff Engineer is **demonstrating thread-safety and concurrency controls**:

- What happens when 1,000 concurrent threads attempt to reserve the same movie seat simultaneously?
- How do you prevent double-charging or race conditions in an inventory system?
- How do you allow high-speed concurrent reads without blocking while ensuring atomic writes?



2. The Readers-Writer Lock (RWLock) Pattern

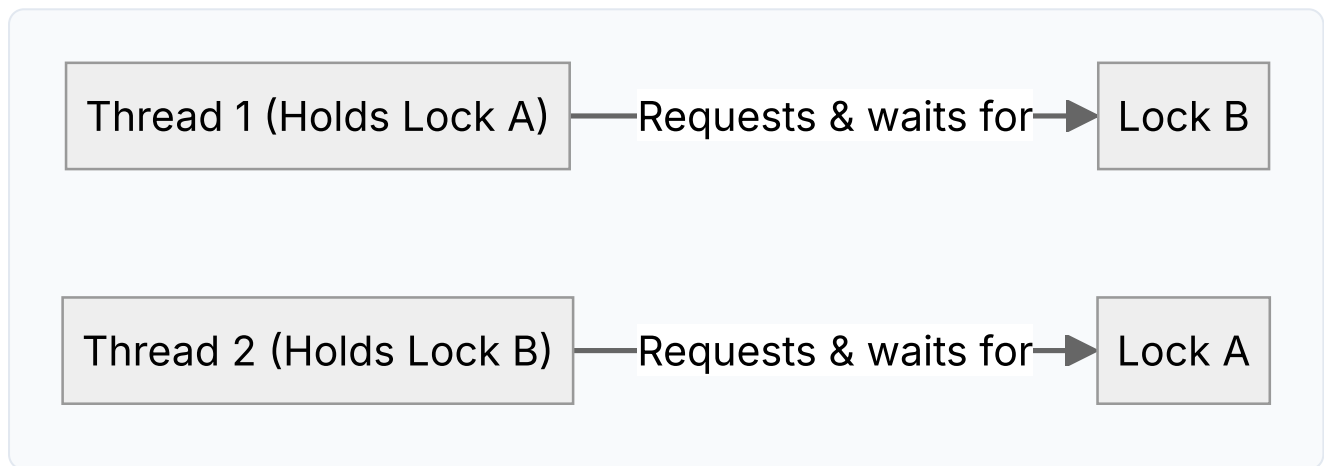
In systems like Bookings, Catalogs, or Config stores, 99% of operations are **Reads**, and only 1% are **Writes**. Using a simple `threading.Lock` forces all readers to wait in a single-file line, destroying throughput. A **Readers-Writer Lock (Shared-Exclusive Lock)** allows:

- **Multiple concurrent Readers** simultaneously.
- **Only ONE Writer** at a time (mutually exclusive with all readers and writers).

Complete Thread-Safe RWLock Implementation in Python

3. Deadlocks & The 4 Coffman Conditions

A **deadlock** is a state where a set of threads are permanently blocked because each holds a lock the other requires.



The 4 Coffman Conditions for Deadlock:

1. **Mutual Exclusion:** Resources cannot be shared; locks must be exclusive.
2. **Hold and Wait:** A thread holds at least one lock while waiting to acquire another.
3. **No Preemption:** Locks cannot be forcibly confiscated from a thread.
4. **Circular Wait:** Thread 1 waits on Thread 2, which waits on Thread 1.

How to Eliminate Deadlocks: Global Lock Ordering

The most reliable way to break the Circular Wait condition is **Global Strict Lock Ordering**: Whenever a thread needs to acquire multiple locks, it **must always acquire them in deterministic alphabetical or ID order**:

Regardless of whether Thread 1 transfers $A \rightarrow B$ and Thread 2 transfers $B \rightarrow A$, both threads attempt to acquire `min(A.id, B.id)` first! Circular wait is mathematically impossible.

4. Double-Checked Locking in Python

Used to initialize expensive singleton resources without paying synchronization lock overhead on every subsequent read access:



Further Reading

- [threading module](#)
- [queue module](#)
- [Wikipedia: Deadlock](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. Why is `count += 1` not thread-safe?
- ▶ 2. What are the four conditions for deadlock?
- ▶ 3. Why prefer a concurrent queue between producers and consumers?
- ▶ 4. When use a read-write lock?